

说明

对于 CH32H417 芯片来说，通常程序默认会加载到 RAM 运行，以便提升性能，但是就整体资源来看，FLASH 960K，RAM（SRAM+ITCM+DTCM）896K，RAM 资源相对少些，而且不能都用于 FLASH 程序的加载运行。所以在此情况下，可以将部分代码通过 Cache 进行指令缓存，可以适当减缓 RAM 资源压力。

适用范围

适用范围	系列
通用 MCU	CH32H417

目录

说明	i
目录	ii
表格索引	ii
图片索引	ii
第 1 章 V5 核 cache 功能简介	1
1.1 Cache 功能简介	1
第 2 章 Cache 使用方法	2
2.1 缓存策略控制 cache	2
2.1.1 相关寄存器及配置位	2
2.1.2 程序配置示例	3
2.2 PMP 通道控制 cache	3
2.2.1 相关寄存器及配置位	4
2.2.2 程序配置示例	5
2.2.3 PMP 通道缓存策略用法总结	5
第 3 章 Cache 性能对比	7
3.1 V5 核 cache 性能 CoreMark 对比	7
3.2 线性测试的 cache 性能对比	8
历史版本	9
声明	10

表格索引

表 1 CACHE 性能对比	8
----------------	---

图片索引

图 2-1 CACHE 缓存策略寄存器位定义	2
图 2-2 CACHE 缓存操作寄存器位定义	3
图 2-3 PMP 模式	3
图 2-4 PMP 地址寄存器位定义	4
图 2-5 PMP 配置寄存器位定义	4
图 2-6 缓存策略 PMP 覆盖寄存器	4
图 3-1 FLASH 运行时 COREMARK 结果	7
图 3-2 FLASH+CACHE COREMARK 结果	7
图 3-3 ITCM COREMARK 结果	7

第1章 V5 核 cache 功能简介

1.1 Cache 功能简介

Icache 是 CPU 内部一块极小的但是运行速度快的内存，通常大小固定，CH32H417 的 Icache 大小为 32K。

Icache 功能启用后，主要工作流程如下：首先 CPU 发生取址请求，不会直接访问慢速地址，而是在 Icache 中进行“查找”并“对比”，如果 Icache 内部地址标签和访问目标地址标签一致则表示缓存命中。若标签不一致则表示缓存未命中，这时候 CPU 流水线会暂停，将进行读取内存请求，并将读出的一块连续数据存储在 Icache 内部索引地址，同时更新标签值。下次 CPU 到相同地址取址时，若能和 Icache 地址标签匹配上，则不需要额外进行内存读取。由于一次读取的 Icache line 通常 64 字节，所以后续的 15 条指令访问都将是连续命中。

CH32H417 的 FLASH 运行速度相对较慢，可以在 FLASH 运行时开启 Icache 功能，可以有效提高 CPU 取址运行速度。

第2章 Cache 使用方法

2.1 缓存策略控制 cache

缓存策略控制 cache 即配置 CACHE_STRTG_CTLR 寄存器，地址 0xBC2。通过配置寄存器对应位功能，使得 CPU 对不同区域进行指令缓存。其优点是操作简单，机器模式下写单个寄存器就能实现。但是控制的地址范围大，如果实际代码超过 32K(Icache 大小)，会经常出现 cache 未命中情况，一定程度上会降低运行效率。

2.1.1 相关寄存器及配置位

CACHE_STRTG_CTLR 寄存器定义如下所示。

图 2-1 cache 缓存策略寄存器位定义

寄存器 bit[24-27]为各个区域的指令缓存控制位，默认都处于开启状态，可以根据实际需求进行配置；bit[1]为指令缓存的使能位，默认关闭状态，清零开启指令缓存功能。

仅仅开启 cache 功能，每次的重新上电会正常运行，因为上电后 cache 的值恢复成随机状态；但是如果是通过软件或者硬件复位，cache 里面的数据将会保留，这时，标记（Tag）和指令数据却是旧程序的。复位后 CPU 取指时，可能错误地“命中”这些旧的指令行，导致 CPU 执行错乱，从而引发非法指令异常或程序跑飞。所以为解决这个问题，可以在开启指令缓存前手动清空缓存内容。寄存器如下，地址 0xBD0，将 bit[2]置 1。

图 2-2 cache 缓存操作寄存器位定义

4.2.3.15 缓存操作寄存器 (OPCACHE_CTLR)

CSR 地址: 0xBD0

位	名称	访问	描述	复位值
[31:5]	Vaddr[27:0]	WO	操作地址或索引信息。	0
[4:3]	Reserved	RO	保留。	0
2	Idx_mode	WO	索引模式 1: 以 vaddr 值为地址信息执行操作; 0: 以 vaddr 值中的索引信息为首地址执行操作。	0
[1:0]	Opcode[1:0]	WO	00: lcache invalidate; 其他: 无效。	0

2.1.2 程序配置示例

```
/* Flush Cache */
li t0, 0x4
csw 0xbd0, t0

/* Enable ICache */
li t0, 0x0E000002
csrw 0xbc2, t0
```

将上述代码放在 startup_ch32h417_v5f.S 文件处，初始化系统的时候配置 cache 功能。其功能首先清空 cache 缓存，其次仅对 0x00000000-0x1fffffff 区域的使能指令缓存功能，并开启该功能。实际可以根据需求设置 0xbc2 的对应功能位，灵活选择需要使能指令缓存的区域。

2.2 PMP 通道控制 cache

PMP 即物理内存保护单元，是 RISC-V 的架构中为了提高系统安全，定义的一套物理地址访问限制，可以为区域内物理内存设置其读、写、执行属性，且能实现区域长度最小 4 字节的保护。QingkeV5 核在 cache 功能配置中，可以利用 PMP 的划分内存保护区域功能，并在所划分的区域通道内使能指令缓存，可以更灵活地使用 cache 功能。

PMP 的内存保护模式有三种，分别是 TOR、NA4、NAPOT，其中 NA4 模式为固定 4 字节保护；NAPOT 为 $2^{(G+2)}$ 字节保护，其中 $G \geq 1$ ；TOR 模式的保护区域大小没有限制，完全通过地址寄存器配置保护通道的边界，所以这种方式相对更灵活。三种模式详细区别如下图。

图 2-3 PMP 模式

2.2.1 相关寄存器及配置位

PMP 地址寄存器共有四个，对于 TOR 模式来说 0 地址与四个 PMP 地址寄存器共能分割出四块区域。地址寄存器如下，需要注意，实际需要将地址右移 2 位后再写入。

图 2-4 PMP 地址寄存器位定义

位	名称	访问	描述	复位值
[31:0]	ADDR0[31:0]	MRW	PMP 设置地址 0 的 bit[33:2], 实际高 2 位未用。	0

PMP 配置寄存器，同样共四组，分别对应不同的通道区域配置，以 pmp0cfg[7:0]为例，如下图。需要配置 bit5 指令缓存功能，以及选择对应的 PMP 模式即 bit[4:3]的 A 值（对应图 2-3 中的 PMP 模式），以及根据需求配置读写执行的权限。

图 2-5 PMP 配置寄存器位定义

[7:0]	pmp0cfg[7:0]	MRW	位	名称	描述	0
			7	L	锁定使能，机器模式下可解锁 0：不锁定； 1：锁定相关寄存器。	
			6	-	保留。	
			5	IC_Str	地址匹配时执行指令缓存策略， 策略值与 CACHE_PMP_OVR 寄存器 对应。	
			[4:3]	A	地址对齐及保护区域范围选择。	
			2	X	可执行属性。	
			1	W	可写入属性。	
			0	R	可读出属性。	

此外还需要配置缓存策略 PMP 覆盖寄存器（CACHE_PMP_OVR），详细位定义如下，对应通道位置 1 后，指令地址受 PMP 对应通道保护时，当前指令可缓存；注意配置完成后任需要开启指令缓存功能，对应寄存器 [CACHE_STRTG_CTLR](#) 的 bit[1]写 0 启用。

图 2-6 缓存策略 PMP 覆盖寄存器

2.2.2 程序配置示例

```
/*Enable the cache to cache the code from _cache_beg to _cache_end */
/*PMP*/
la t0, _cache_beg
srli t0, t0, 2
csrw pmpaddr0, t0

la t0, _cache_end
srli t0, t0, 2
csrw pmpaddr1, t0

/*PMP channel 1*/
li t0, 0x10
csrw 0xbc3, t0

li t0, 0xAD00 /*lock*/
csrw 0x3a0, t0

/* Flush Cache */
li t0, 0x4
csrw 0xbd0, t0

/* Enable ICache */
li t0, 0xF000002
csrw 0xbc2, t0
```

2.2.3 PMP 通道缓存策略用法总结

2.2.2 节程序示例中，cache_beg 和 _cache_end 地址标签值，是通过 LD 文件定义并获取的，是实际分配 FLASH 段中已使用区域的首末地址。如下，比如我们在 LD 中划分一块 FLASH1 区域，起始地址 0x08070000，大小 32K。

```
.text1 :
{
    . = ALIGN(4);
    KEEP(*(SORT_NONE(.text1)))
    PROVIDE( _cache_beg = .);
    *(.cache);
    *(.cache.*);
    PROVIDE( _cache_end = .);
    . = ALIGN(4);
} >FLASH1 AT>FLASH1
```

函数前需要加上 __attribute__((section(".cache")))，编译后存放在 FLASH1 中。如果实际代码量超过 32K，编译会报错提示，这时候只需要调整 FLASH1 分配的大小即可。需要注意这时候指令缓存区域的大小已经超过了 cache 的总容量，因此 cache 整体性能会有下降。

对于 2.2.2 节程序示例中，cache_beg 和 _cache_end 地址标签值，我们同样可以更换为实际 FLASH 地址值。比如我们在 LD 中划分一块 FLASH1 区域，起始地址 0x08070000，大小 32K，代码即可如下修改

```
la t0, 0x00070000
srli t0, t0, 2
csrw pmpaddr0, t0

la t0, 0x00078000
srli t0, t0, 2
csrw pmpaddr1, t0
```

LD 中新增一个段描述，用于将函数存放在这块 FLASH 中，函数前需要加上 `__attribute__((section(".cache1")))`

```
.text1 :
{
    . = ALIGN(4);
    KEEP(*(SORT_NONE(.text1)))
    *(.cache1);
    *(.cache1.*);
    . = ALIGN(4);
} >FLASH1 AT>FLASH1
```

这种方式预先分配了 32K 大小的 FLASH 用于指令加载，如果实际代码量超过 32K,编译会报错提示，这时候可以调整分配的 FLASH1 的大小，以及 pmpaddr1 寄存器的值。

第3章 Cache 性能对比

3.1 V5 核 cache 性能 CoreMark 对比

指令存储在 FLASH 中，跑 CoreMark 结果对比如下，同样 GCC15 标准下。

指令存储在 FLASH 中的 CoreMark 结果如下 0.327/MHz

图 3-1 FLASH 运行时 Coremark 结果

```
MCU:CH417
V5SystemClk:75000000Hz
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 764625858
Total time (secs): 10.195011
Iterations/Sec : 24.521796
Iterations : 250
Compiler version : GCC15.2.0
Compiler flags : -Ofast
Memory location : Static
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0xf8b3
Correct operation validated. See README.md for run and reporting
rules.
CoreMark 1.0 : 24.521796 / GCC15.2.0 -Ofast / Static
If working at 1MHz,CoreMark 1.0 : 0.326957/MHz
```

指令存储在 FLASH 中的并且使能了指令缓存后的 CoreMark 结果如下 6.672/MHz

图 3-2 FLASH+Cache Coremark 结果

```
MCU:CH417
V5SystemClk:75000000Hz
2K performance run parameters for coremark.
CoreMark Size : 666
Total ticks : 824292378
Total time (secs): 10.990565
Iterations/Sec : 500.429229
Iterations : 5500
Compiler version : GCC15.2.0
Compiler flags : -Ofast
Memory location : Static
seedcrc : 0xe9f5
[0]crclist : 0xe714
[0]crcmatrix : 0x1fd7
[0]crcstate : 0x8e3a
[0]crcfinal : 0x33ff
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 500.429229 / GCC15.2.0 -Ofast / Static
If working at 1MHz,CoreMark 1.0 : 6.672389/MHz
```

指令存储在 ITCM 紧耦合区域的 CoreMark 结果如下 6.802/MHz

图 3-3 ITCM Coremark 结果

从对比结果看 cache 对缓存在 FLASH 中指令加载运行性能提升很大，接近紧耦合 ITCM 区域。测试代码容量未超过 32K，如果实际缓存内容超过 32K，cache 性能会有所降低。

3.2 线性测试的 cache 性能对比

将 12K 的函数放在 FLASH 中，函数内容为 nop 展开，线性无循环执行。

表 1 cache 性能对比

函数执行次数	FLASH 中运行, Cache 关(us)	FLASH 中运行, Cache 关(us)
1	253.35	1619.32
2	509.26	1627.05
3	763.28	1634.77
.....		
10	2545.90	1688.81
.....		
20	5093.08	1766.01

从表中的数据可以看出，FLASH 中运行速度比较平均，cache 首次加载的时间较长，但是运行速度是 FLASH 中的 30 倍，如果实际代码量不超过总的 cache 大小或者不会频繁加载 cache 时，当执行次数越多时，cache 整体性能表现将越好。

历史版本

更新内容

日期	版本	变更内容
2026/04/24	V1.0	初版发行

声明

本手册版权所有为南京沁恒微电子股份有限公司（Copyright © Nanjing Qinhang Microelectronics Co., Ltd. All Rights Reserved），未经南京沁恒微电子股份有限公司书面许可，任何人不得因任何目的、以任何形式（包括但不限于全部或部分地向任何人复制、泄露或散布）不当使用本产品手册中的任何信息。

任何未经允许擅自更改本产品手册中的内容与南京沁恒微电子股份有限公司无关。

南京沁恒微电子股份有限公司所提供的说明文档只作为相关产品的使用参考，不包含任何对特殊使用目的的担保。南京沁恒微电子股份有限公司保留更改和升级本产品手册以及手册中涉及的产品或软件的权利。

参考手册中可能包含少量由于疏忽造成的错误。已发现的会定期勘误，并在再版中更新和避免出现此类错误。